pycognition: Дуальная когнитивная архитектура, интегрирующая детерминированное процедурное исполнение и делиберативный синтез языковых моделей

Олег Рябцев
Referat Forschung, Nachwuchsförderung und Transfer (RFNT), Universität Vechta, Германия
ORCID: 0009-0003-7848-6418 | Лицензия: LGPL-3.0-or-later

АННОТАЦИЯ

В настоящей работе представлена архитектура **pycognition** — открытый программный комплекс, формализующий положения психологической теории двойного процесса (Dual-Process Theory) для класса автономных программных агентов. Доминирующие агентные парадигмы направляют поток детерминированных и циклических подзадач исключительно через удаленные API больших языковых моделей (LLM), что приводит к нарастанию задержки отклика, квадратичному росту затрат на токены и стохастическому расхождению результатов. Предлагаемая архитектура преодолевает эти ограничения путем структурного разделения вычислений на два контура: быстрый процедурный исполнитель (Система 1, детерминированные Python-скрипты) и глубокий делиберативный планировщик (Система 2, многошаговый логический вывод frontier-моделей). Интеграция контуров обеспечивается механизмом кристаллизации навыков (Skill Crystallization), который автономно транслирует успешные цепочки рассуждений Системы 2 в компилируемый код, верифицирует его безопасность через аудит абстрактного синтаксического дерева (AST) и тестирует в изолированной песочнице `pytest` перед горячей загрузкой в память. Эмпирическая оценка на гетерогенной выборке из 100 сложных задач подтвердила надежность защитного барьера (отсев 36,2% дефектных генераций) и позволила построить реалистичную трёхъярусную модель производительности. В отличие от наивных сопоставлений, модель явно учитывает накладные расходы на автономную разработку скрипта (в среднем 7 192,20 мс). Точка безубыточности (break-even point) для отдельного навыка наступает на 4-м вызове при идеальном синтезе и на 5-м вызове с учетом брака песочницы, после чего достигается асимптотическое ускорение в 11 326,7 раза. На уровне всей системы время ответа на алгоритмические задачи сократилось на 44,7% (155,46 с против 281,13 с), а совокупный расход токенов уменьшился на 38,0% с выходом на 71,4% процедурного покрытия в четвертом квартиле нагрузки.

Ключевые слова: когнитивные архитектуры, автономные агенты, дуальный процесс, синтез программного кода, AST-валидация, амортизированный вывод, JIT-компиляция, точка безубыточности, pycognition.

1. ВВЕДЕНИЕ

Широкое внедрение автономных интеллектуальных агентов на базе больших языковых моделей (LLM) выявило фундаментальное архитектурное противоречие между гибкостью генерации естественного языка и вычислительной эффективностью выполнения детерминированных алгоритмов. Современные промышленные агентные фреймворки (ReAct [6], Plan-and-Solve [9], AutoGen [10]) используют стохастическую авторегрессионную модель в роли безальтернативного универсального вычислителя. Типовые алгоритмические трансформации, строковый парсинг логов, вычисление геодезических метрик расстояния и финансовая математика повторно транслируются в текстовые запросы к облачным API наряду с задачами комплексного стратегического планирования.

Подобная организация вычислительного процесса порождает три критических системных барьера, препятствующих внедрению агентов в высоконагруженные корпоративные инфраструктуры:

- **Экономическая неэффективность (Token Overhead):** Линейная и квадратичная аккумуляция затрат на токены при обработке повторяющихся задач. Необходимость сериализации контекста и истории рассуждений в промпт приводит к лавинообразному расходу вычислительного бюджета облачных API.
- **Критическая задержка отклика (Inference Latency):** Круговой сетевой цикл обращения к коммерческим API составляет от 1 500 до 5 000 мс на транзакцию. Данный показатель неприемлем для систем интерактивного взаимодействия и реактивного управления в реальном времени.
- **Стохастическая ненадежность (Non-deterministic Drift):** Вероятностная природа авторегрессионного декодирования не предоставляет формальных гарантий инвариантности. Функция, корректно решенная моделью на шаге k, может вернуть некорректный результат или синтаксическую ошибку на шаге $k + 1$.

В когнитивной психологии человеческий интеллект преодолевает эти барьеры посредством дуальной организации мышления, описанной Д. Канеманом [1] и формализованной Дж. Эвансом и К. Становичем [2]: **Система 1** (быстрое, автоматическое, бессознательное выполнение наработанных процедурных навыков) и **Система 2** (медленное, осознанное, аналитическое рассуждение в условиях неопределенности). Ключевым свойством естественного интеллекта является *процедурная консолидация опыта*: аналитические цепочки рассуждений Системы 2 постепенно компилируются в автоматизированные двигательные и логические навыки Системы 1.

Архитектура **pycognition** реализует указанный нейросимволический переход для автономных программных агентов. Фреймворк функционирует как обучающийся Just-In-Time (JIT) компилятор: сталкиваясь с новой задачей, агент инвестирует ресурсы Системы 2 в синтез детерминированного кода на языке Python, верифицирует его безопасность и динамически регистрирует в процедурной памяти. Настоящее исследование формулирует ответы на три исследовательских вопроса:

RQ1: Какова математическая граница метакогнитивного арбитража, обеспечивающая баланс между стоимостью обучения и точностью исполнения при гетерогенном потоке задач?
RQ2: Каким образом построить изолированный конвейер проверки синтаксиса и динамического тестирования, гарантирующий безопасность и надежность сгенерированного кода перед его фиксацией в памяти?
RQ3: Какова эффективная амортизированная динамика системы (точка безубыточности с учетом затрат времени на разработку кода) в потоке повторяющихся и уникальных задач?

2. СОСТОЯНИЕ ПРОБЛЕМЫ И СМЕЖНЫЕ РАБОТЫ

2.1. Агентные парадигмы и вычислительное трение

Эволюция агентных систем на основе шаблона ReAct (Yao et al. [6]) и структурированного планирования Plan-and-Solve (Wang et al. [9]) закрепила циклический протокол «рассуждение — действие — наблюдение». В многоагентных средах (AutoGen [10]) координация осуществляется путем взаимной генерации сообщений на естественном языке. Это приводит к квадратичному росту вычислительных затрат на механизме внутреннего внимания (self-attention) и усугубляет сетевую задержку.

2.2. Когнитивные архитектуры и амортизированный вывод

Концептуальное разделение декларативной и процедурной памяти исследовалось в классических архитектурах ACT-R (Anderson [3]) и Soar (Laird [4]). В Soar механизм *chunking* компилировал результаты разрешения тупиковых состояний логического вывода в постоянные продукционные правила. В вероятностном программировании концепция амортизированного вывода (Gershman & Goodman [15]) доказала целесообразность замены трудоемкого стохастического поиска детерминированным кэшированным оператором. В современных языковых моделях подход Toolformer [8] обеспечивает вызов внешних инструментов, однако каталог функций остается статическим и требует ручного проектирования инженером.

2.3. Программный синтез и верификация кода

Развитие моделей генерации кода (Codex [12], DeepSeek-Coder [13]) доказало принципиальную возможность автономного создания рабочих модулей на Python. Агент Voyager (Wang et al. [11]) продемонстрировал непрерывное накопление программных навыков в игровой среде Minecraft. Однако в нем отсутствовали формальный аудит абстрактного синтаксического дерева (AST), строгая изоляция времени исполнения и автоматизированная генерация проверочных тестов [14], что затрудняло интеграцию концепции в корпоративные программные комплексы.

Таблица 1. Сравнительный анализ архитектурных парадигм автономных агентов

Критерий сравнения	**Pure Prompting (Zero-Shot)**	**ReAct / AutoGen Multi-Agent**	**Static Tool-Use (LangChain)**	**pycognition (Dual-Process)**
Задержка исполнения (Latency)	Высокая (1 500–3 000 мс)	Критическая (> 5 000 мс)	Смешанная (зависит от LLM)	Субмиллисекундная (< 1 мс для Системы 1)
Расход токенов (Token Cost)	Линейный на каждый запрос	Квадратичный рост контекста	Умеренный	Нулевой для скомпилированных задач
Детерминизм вычислений	Низкий (стохастический)	Низкий (стохастический)	Частичный (фиксированные тулы)	Абсолютный (Python Unit-Tested)
Расширение базы навыков	Отсутствует	Отсутствует	Ручное кодирование инженером	Автономная кристаллизация (AST + Pytest)
Изоляция безопасности (Sandbox)	Не требуется	Ограниченная (Raw Exec)	Определяется разработчиком	AST-фильтрация + изолированный Subprocess

3. МЕТОДОЛОГИЯ И СИСТЕМНАЯ АРХИТЕКТУРА

3.1. Формализация когнитивного пространства

Входящая вычислительная задача T формализуется как четырехэлементный кортеж:

$$T = \langle \mathrm{id}, p, x, \mu \rangle$$

где id — уникальный идентификатор задачи (UUIDv4); p — строковый промпт запроса на естественном языке; x — структурированный словарь входных параметров (payload); μ — метаданные операционного окружения.

Процедурная память определяется множеством зарегистрированных детерминированных навыков $\Sigma = \{\sigma_1, \sigma_2, \ldots, \sigma_m\}$, где каждый навык σ_i представляет собой упорядоченную пару:

$$\sigma_i = \langle \mathrm{Sig}_i, h_i \rangle$$

где Sig_i — строго типизированная интерфейсная спецификация, а $h_i : X \to Y$ — скомпилированная и динамически загруженная через `importlib` Python-функция в модуле `ProceduralRunner`.

3.2. Функция полезности и метакогнитивный арбитраж

Модуль `MetaArbiter` выбирает целевой когнитивный контур $a \in \{S_1, S_2\}$ путем максимизации функции ожидаемой полезности:

$$U(a \mid T) = E[Q(a,T)] - \lambda_c \cdot C(a) - \lambda_\ell \cdot L(a)$$

Семантическое соответствие входящей задачи базе навыков в `VectorStoreMemory` вычисляется через максимум косинусного сходства нормализованных TF-векторов (с автоматическим расщеплением `snake_case` идентификаторов и индексацией триггерных промптов):

$$\gamma(T, \Sigma) = \max_{\sigma_i \in \Sigma} \mathrm{sim}(e(p), e(\mathrm{Sig}_i))$$

Решающее правило маршрутизации определяется как:

$$\mathrm{Route}(T) = \begin{cases} \langle S_1, \sigma_i^* \rangle, & \text{при } \gamma(T,\Sigma) \geq \theta \\ \langle S_2, \bot \rangle, & \text{при } \gamma(T,\Sigma) < \theta \end{cases}$$

где $\theta = 0.55$ — откалиброванный порог уверенности.

3.3. Конвейер кристаллизации навыков (Skill Crystallization)

При успешном решении задачи делиберативным контуром запускается оператор кристаллизации Ψ:

$$\Psi : \langle T, y_{S_2}, C \rangle \xrightarrow{\mathrm{AST}_{\mathrm{safe}} \wedge \mathrm{Test}_{\mathrm{pass}}} \sigma_{\mathrm{new}} \in \Sigma$$

Статический валидатор `ASTSecurityValidator` гарантирует отсутствие опасных конструкций в дереве разбора кода:

$$\mathrm{AST}_{\mathrm{safe}}(\mathrm{code}) \Leftrightarrow (\mathrm{Imports} \cap B_{\mathrm{mod}} = \emptyset) \wedge (\mathrm{Calls} \cap B_{\mathrm{func}} = \emptyset)$$

где $B_{\mathrm{mod}} = \{\mathrm{os}, \mathrm{sys}, \mathrm{subprocess}, \ldots\}$, $B_{\mathrm{func}} = \{\mathrm{eval}, \mathrm{exec}, \ldots\}$, а динамическая проверка выполняется запуском `pytest` в изолированном подпроцессе с таймаутом $t = 15$ с.

4. РЕАЛИЗАЦИЯ И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА

4.1. Программный комплекс и стендовое окружение

Фреймворк **pycognition** реализован на языке Python 3.10+ в модульной топологии `src-layout` под управлением `hatchling` (PEP 621) и лицензирован на условиях **LGPL-3.0-or-later**. Модели данных

типизированы через `Pydantic v2`. В качестве делиберативной модели Системы 2 выступал сервис `DeepSeek-V3` (`deepseek-chat`) через `httpx`.

4.2. Нагрузочный профиль эксперимента (N = 100)

Экспериментальный поток сформирован из $N = 100$ задач: 85 алгоритмических вычислений над 12 классами по распределению Ципфа (дистанция гаверсинусов, расстояние Левенштейна, агрегация транзакций, извлечение IPv4, аннуитетный расчет, детекция выбросов z-score, скользящие средние EMA, скобочные последовательности, частотный анализ слов, умножение матриц, RLE-сжатие, расчет рабочих дней) и 15 открытых концептуальных запросов CoT.

4.3. Реалистичная трёхъярусная модель временной эффективности

Строгий анализ производительности требует отказа от иллюзорного прямого сопоставления миллисекундного выполнения скрипта с вызовом нейросети без учета времени, затраченного на синтез этого скрипта. Модель эффективности pycognition формализуется в трех плоскостях:

- **Ярус 1. Маржинальная задержка и асимптотическое ускорение:** После компиляции скрипт Системы 1 выполняется в среднем за $\overline{L}_{S_1} = 0.292$ мс ($\min = 0.10$ мс, $\max = 0.70$ мс). Прямой вызов LLM для аналогичной алгоритмической задачи требует в среднем $L_{\text{base}} = 3\,307.41$ мс. Предельное *маржинальное ускорение* скомпилированного скрипта составляет:

$$S_{\text{marginal}} = \frac{L_{\text{base}}}{\overline{L}_{S_1}} = \frac{3\,307.41 \text{ мс}}{0.292 \text{ мс}} \approx 11\,326.7\times \qquad (S_{\text{overall,marginal}} = \frac{5\,580.36 \text{ мс}}{0.292 \text{ мс}} \approx 19\,103.9\times)$$

- **Ярус 2. Затраты на разработку скрипта и точка безубыточности отдельного навыка:** На первичном холодном старте агент не только решает задачу (3 307.41 мс), но и инициирует конвейер разработки скрипта (генерация кода, AST-проверка, запуск тестов в песочнице), требующий в среднем $L_{\text{cryst}} = 7\,192.20$ мс. Суммарное время холодного старта составляет $L_{\text{cold}} = 3\,307.41 + 7\,192.20 = 10\,499.60$ мс (~10,50 с).
  Условие окупаемости инвестиций времени для k повторений задачи:

$$T_{\text{pycognition}}(k) = L_{\text{base}} + L_{\text{cryst}} + (k-1)\cdot\overline{L}_{S_1} \le k\cdot L_{\text{base}} \quad \Longrightarrow \quad k \ge 1 + \frac{L_{\text{cryst}} - \overline{L}_{S_1}}{L_{\text{base}} - \overline{L}_{S_1}}$$

  Численный расчет показывает:

$$k_{\text{ideal}} \ge 1 + \frac{7\,192.20 - 0.292}{3\,307.41 - 0.292} \approx 3.175 \Longrightarrow k = 4 \text{ вызова}$$

  С учетом 36,2% отсева в песочнице эффективные затраты на один принятый навык составляют $L_{\text{cryst,eff}} = (47/30)\cdot 7\,192.20 = 11\,267.78$ мс, что сдвигает безубыточность до $k_{\text{eff}} \ge 4.407 \Longrightarrow k = 5$ вызовов.
  Функция эффективного амортизированного ускорения от числа повторений:

$$S_{\text{skill}}(k) = \frac{k\cdot 3\,307.41}{10\,499.60 + (k-1)\cdot 0.292}$$

  Динамика окупаемости: при $k = 1$ ускорение составляет $0.31\times$ (замедление из-за компиляции); при $k = 2$ — $0.63\times$; при $k = 4$ — $1.26\times$ (безубыточность); при $k = 10$ — $3.15\times$; при $k = 50$ — $15.73\times$; при $k = 100$ — $31.41\times$.

- **Ярус 3. Общесистемная динамика на потоке N = 100 задач:**
  В базовом сценарии 100 вызовов LLM заняли бы **471,66 с** (281,13 с на алгоритмы + 190,53 с на концепты). В pycognition при асинхронной разработке скрипта время ответа на алгоритмические задачи составило **155,46 с против 281,13 с (ускорение в 1,81 раза, экономия 44,7% времени)**.
  В синхронном режиме с учетом всех 47 циклов разработки скриптов (338,03 с) общее время составило 493,49 с на алгоритмах (684,03 с суммарно). Кварцевая динамика подтверждает переход: в Q1 синхронное ускорение составило 0,38x, в Q2 — 0,60x, в Q3 — 0,60x, а в Q4 достигло **0,96x (практически безубыточность внутри квартиля при ускорении самого ответа в 2,84x)**.
  Полный системный горизонт безубыточности при сохранении темпа Q4 достигается при:

$$N^{*} = 100 + \frac{212\,363 \text{ мс}}{3\,307.11 \text{ мс} \cdot 0.60} \approx 207 \text{ задач}$$

4.4. Лонгитюдная динамика амортизации токенов

В отличие от времени, токены амортизируются мгновенно. Фактический расход составил **53 978 токенов против 87 061 токена (экономия 38,0%, или 33 083 токена)**. При этом доля процедурного выполнения выросла с 14,3% в Q1 до **71,4% в Q4**:

$$\text{Quartile Ratio}_{S_1}(q) = \frac{N_{S_1}(q)}{N_{\text{algo}}(q)}: \quad Q_1 = \frac{3}{21}\,(14.3\%), \quad Q_2 = \frac{10}{22}\,(45.5\%), \quad Q_3 = \frac{10}{21}\,(47.6\%), \quad Q_4 = \frac{15}{21}\,(71.4\%)$$

Таблица 2. Реалистичные показатели эффективности pycognition (N = 100 задач)

Показатель / Метрика	Чистый LLM-контур (Baseline)	pycognition Dual-Process	Реалистичный эмпирический эффект
Маржинальная задержка Системы 1	3 307,41 мс (алго)	0,292 мс (Система 1)	Субмиллисекундный отклик
Задержка разработки скрипта (Crystallization)	0 мс (нет синтеза)	7 192,20 мс (средняя)	Инвестиции в JIT-компиляцию
Полная задержка холодного старта	3 307,41 мс	10 499,60 мс	Временной штраф обучения (0,31x)
Точка безубыточности навыка (Break-Even)	Не применима	k = 4 (идеал) / k = 5 (с браком)	Окупаемость за 4–5 вызовов
Маржинальное ускорение скрипта	1,0x	11 326,7x (алго) / 19 103,9x (общ.)	Асимптотический предел
Время ответа на 85 алгоритмических задач	281,13 с	155,46 с (асинхронно)	Ускорение в 1,81 раза (-44,7% времени)
Синхронное время (с учетом разработки)	281,13 с	493,49 с	Инвестиционная фаза при N=100
Горизонт системной безубыточности N*	Не применим	N* ≈ 207 задач	Полная окупаемость кластера
Суммарный расход токенов (100 задач)	87 061 токенов	53 978 токенов	-38,0% немедленной экономии
Доля Системы 1 в 4-м квартиле (Q4)	0,0%	71,4% (15 из 21 алгоритмических)	Устойчивая консолидация
Результативность верификации в песочнице	Не применима	63,8% (30 принято / 17 отсеяно)	100% надежность памяти Системы 1

5. ОБСУЖДЕНИЕ И ОГРАНИЧЕНИЯ

5.1. Анализ барьера верификации: отказы песочницы как фундаментальный защитный инвариант

Из 47 обращений к Системе 2 успешно принято 30 модулей (63,8%), а 17 кандидатов (36,2%) отсеяны песочницей. Анализ выявил:

- **Степени свободы (6 отказов):** в z-score модель путала смещенную дисперсию (N) и несмещенную с поправкой Бесселя ($N-1$), либо допускала деление на ноль.
- **Краевые условия IPv4 (4 отказа):** регулярные выражения пропускали октеты > 255.
- **Синтаксис и округление (7 отказов):** лишние символы в тестах скобок и несовпадения в 5-м знаке float.

Отсев предотвратил скрытые ошибки: все 38 последующих вызовов Системы 1 завершились со статусом `SUCCESS`.

5.2. Реалистичная оценка накладных расходов компиляции

Признание времени разработки скрипта (7,19 с) устраняет спекулятивные оценки. В агентных архитектурах компиляция должна выполняться асинхронно в фоновом режиме, отдавая пользователю первичный ответ модели и подключая Систему 1 начиная со второго вызова.

5.3. Семантическая фрагментация и кросс-языковое разбавление TF-векторов

30 успешных кристаллизаций породили 20 имен функций (например, `run_length_encode` компилировался 4 раза). Причины:

- **Синонимия промптов:** без нейросетевых эмбеддингов формулировки-синонимы дают сходство ниже $\theta = 0.55$.
- **Кросс-языковое разбавление нормы:** русскоязычные метаданные `description` раздували норму $\|v_{\text{skill}}\|_2$ при англоязычных промптах задач в формуле:

6. ЗАКЛЮЧЕНИЕ И ПЕРСПЕКТИВЫ

6.1. Основной вклад

В работе обоснована и верифицирована архитектура **pycognition** с реалистичной трехъярусной оценкой JIT-компиляции. Доказано, что первичные затраты на разработку скриптов (7,19 с) окупаются за 4–5 обращений к задаче, обеспечивая маржинальное ускорение в 11 326,7 раза, экономию 38,0% токенов и ускорение чистого времени ответа в 1,81 раза.

6.2. Дорожная карта исследований

1. Самоисцеляющиеся циклы синтеза по трейсбекам `pytest` для подъема процента кристаллизации с 63,8% до >90%.
2. Плотные многоязычные векторные эмбеддинги для дедупликации навыков.
3. Адаптивный арбитраж на базе контекстуальных многоруких бандитов.
4. Федеративные реестры для децентрализованного обмена подписанными модулями.

7. СПИСОК ЛИТЕРАТУРЫ

[1] D. Kahneman, *Thinking, Fast and Slow*. New York, NY, USA: Farrar, Straus and Giroux, 2011.

[2] J. S. B. T. Evans and K. E. Stanovich, "Dual-process theories of higher cognition: Advancing the debate," *Perspectives on Psychological Science*, vol. 8, no. 3, pp. 223–241, 2013.

[3] J. R. Anderson, *How Can the Human Mind Occur in the Physical Universe?* New York, NY, USA: Oxford University Press, 2007.

[4] J. E. Laird, *The Soar Cognitive Architecture*. Cambridge, MA, USA: MIT Press, 2012.

[5] B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman, "Building machines that learn and think like people," *Behavioral and Brain Sciences*, vol. 40, p. e253, 2017.

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[7] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS*, 2022.

[8] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.

[9] L. Wang, W. Xu, Y. Lan, Z. Shen, E. Lan, F. Zhu, and E.-P. Lim, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proc. ACL*, 2023.

[10] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. KDD*, 2024.

[11] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," in *Proc. NeurIPS*, 2023.

[12] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, "Evaluating large language models trained on code," *arXiv:2107.03374*, 2021.

[13] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, R. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, *et al.*, "DeepSeek-Coder: When the large language model meets programming – The rise of code intelligence," *arXiv:2401.14196*, 2024.

[14] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proc. ICSE*, 2023.

[15] S. J. Gershman and N. D. Goodman, "Amortized inference in probabilistic reasoning," in *Proc. CogSci*, 2014.

[16] A. d'Avila Garcez and L. C. Lamb, "Neurosymbolic AI: The 3rd wave," *Artificial Intelligence Review*, vol. 56, pp. 12387–12406, 2023.